

MANUAL DE EJECUCION Y DEMOSTRACION EN VIVO

TRADING ROI EXPLAINER & SIMULATOR API (GEMINI 3.5 & OPENTELEMETRY)

Autor: Marcelo Gilardoni

Rol: Solutions Architect / DevOps

Este manual describe el procedimiento exacto paso a paso para levantar, probar y demostrar en vivo todas las capacidades técnicas de tu Trabajo Final de Integración. Detalla el funcionamiento de la petición normal inicial, el comportamiento de resiliencia ante fallos (Opción A), la ejecución de pruebas y percentiles de latencia (Opción B), y el monitoreo avanzado en la nube con Langfuse (Opción C).

1. Levantamiento del Servidor FastAPI (Punto de Partida)

Para iniciar la aplicación, abre tu terminal de PowerShell en Windows y ejecuta el comando literal:

```
& "C:\OpenCode\sesion_3\.venv\Scripts\uvicorn.exe" main:app --reload --host 127.0.0.1 --port 8000 --app-dir "C:\OpenCode\1-integracion-tbf\codigo"
```

Explicación de lo que ocurre:

- Se carga el entorno virtual de Python con todas las dependencias oficiales.
- El servidor Uvicorn arranca de forma asincrónica y se queda escuchando peticiones locales en la URL: `http://127.0.0.1:8000`
- OpenTelemetry (OTel) se inicializa de forma automática correlacionando logs con `TraceID/SpanId`.

2. Ejecución de la Petición Normal Inicial (Swagger)

1. Abre en tu navegador de internet la interfaz Swagger: `http://127.0.0.1:8000/docs`
2. Haz clic en 'Try it out' en el endpoint `POST /api/v1/trading/explain`.
3. Inserta la clave de seguridad 'trading-secret-key-123' en la casilla 'X-API-Key'.
4. En el Request Body, envía el siguiente JSON conteniendo tus datos en claro:

```
{
  "user_name": "Marcelo Gilardoni",
  "message": "Tengo $1500 USD y quiero simular una inversión en BTC si sube a $80000"
}
```

5. Haz clic en el gran botón azul 'Execute' y observa el resultado HTTP 200 OK.

Explicacion Detallada del Resultado de la Peticion Normal:

- Enmascaramiento PII Activo: El adaptador del backend intercepto tu nombre y lo enmascaro como 'M***** G*****' en la respuesta final de Gemini. Tu identidad nunca sale de tu servidor web.
- Extraccion de Parametros: El sistema limpio el texto del mensaje y extrajo unicamente las variables necesarias (activo='BTC', monto=1500.0, precio_meta=80000.0) para prevenir ataques de prompt injection.
- Logica Matematica Exacta: El adaptador realizo los calculos en frio deduciendo la comision (0.5% = \$7.50 USD) y calculo de forma automatica un ROI del 1.36%, guardando la simulacion de forma limpia en mock_db.json.
- Auditoria de Red: Gemini 3.5 redacto de manera perfecta el informe financiero en Markdown en un promedio de 3 segundos de procesamiento total (x-process-time).

3. Simulacion de Falla en Base de Datos (Opcion A - ACID)

Pasos de ejecucion en vivo:

1. Ve al archivo 'C:\OpenCode\1-integracion-tbflcodigo\database\mock_db.json' en Windows.
2. Haz clic derecho -> Propiedades -> Marca 'Solo lectura' (Read-only) -> Aceptar.
3. Regresa a tu navegador o ejecuta el script de demostracion de consola:
python codigo/scripts/demo_gemini_streaming.py
4. Observa la respuesta HTTP 503 Service Unavailable controlada.

Explicacion Detallada de la Falla y Resiliencia (Opcion A):

- Aborto Seguro y Control de Costos: Al fallar la escritura en disco por falta de permisos, el adaptador detuvo el hilo de ejecucion antes de llamar a Gemini 3.5. Esto previene facturar costos de red inútiles y asegura la integridad transaccional (si no se puede guardar, no se ejecuta).
- Respuesta Consistente: El cliente recibe un JSON ordenado HTTP 503 informando que la operacion fallo por consistencia de base de datos y que debe reintentar.
- Diagnostico con OpenTelemetry: El Span 'simulate_transaction' se marco con estado 'ERROR' y registro la excepcion 'PermissionError', imprimiendo la alerta en la consola del servidor.
- *(Restauracion): No olvides desmarcar la casilla de Solo lectura para continuar con las pruebas.*

4. Pruebas Unitarias y Medidores de Latencia (Opcion B)

Pasos de ejecucion en vivo:

1. Ejecutar las 6 pruebas de Pytest locales asocian la salud de la API, claves y enmascaramiento:
`python -m pytest codigo/tests/test_trading_api.py -v`
- Observaras: '6 passed in less than 2s'. Las pruebas corren deterministamente offline sin costos.
2. Ejecutar el script de Benchmark de Latencias (10 llamadas consecutivas):
`python codigo/scripts/benchmark_latencia.py`
- Se realizara una simulacion de carga de 10 peticiones secuenciales.

Explicacion Detallada de Metricas (Opcion B):

- El benchmark reportara la mediana p50 (~20.97 ms) y el percentil p95 (~28.86 ms) de la velocidad de tu framework FastAPI y persistencia local.
- Los resultados son cientificos, medidos de forma real e historizados en 'database/benchmark_results.json'.

5. Observabilidad Cognitiva y Datasets en la Nube (Opcion C)

Pasos de ejecucion en vivo:

1. Crear el Dataset 'trading_agent_evaluation' en tu cuenta de Langfuse Cloud:
`python codigo/scripts/create_langfuse_dataset.py`
2. Correr la evaluacion asincrona en vivo de tus 3 casos de prueba con Gemini 3.5:
`python codigo/scripts/run_langfuse_evaluation.py`
3. Abre tu navegador web en: <https://us.cloud.langfuse.com>

Explicacion Detallada de Trazabilidad LLM Ops (Opcion C):

- Muestra al docente la pestaña 'Datasets' y enseña los 3 casos de prueba cargados.
- Muestra la pestaña 'Runs' donde se compara de forma automatica tu Entrada, la Respuesta del LLM y el Resultado Esperado de Referencia. Resalta el enmascaramiento (User ID: M**** G****) y los costes estimados calculados por Langfuse para auditoria de produccion.

Manual Tecnico Completo - Marcelo Gilardoni | Solutions Architect & DevOps.